

Automatic Differentiation (Part I)

Jonathon Hare

Much of this material is based on this blog post: <https://ruffwind.com/2016-12-30/reverse-mode-automatic-differentiation>

What is Automatic Differentiation (AD)?

To solve optimisation problems using gradient methods we need to compute the gradients (derivatives) of the objective with respect to the parameters.

- In neural nets we're talking about the gradients of the loss function, $\mathcal{L}$ with respect to the parameters θ : $\nabla_{\theta}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$
- AD is important - it's been suggested that "Differentiable programming" could be the term that ultimately replaces deep learning¹.

How can you compute derivatives?

There are three ways to compute derivatives:

- Symbolically differentiate the function with respect to its parameters
 - by hand
 - using a CAS
- Make estimates using finite differences
- Use Automatic Differentiation

Problems

Static - can't "differentiate algorithms"

Numerical Differentiation

Suppose we can evaluate $f(x)$, but do not know how to differentiate it.

The basic idea

Perturb the input by a small amount h , observe how much the output changes, and use the slope of the resulting secant line:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

This is a **one-sided finite difference**: it replaces a limit with two function evaluations at nearby points.

What error have we introduced?

Taylor-expand $f(x+h)$ around x :

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3)$$

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2).$$

¹<http://forums.fast.ai/t/differentiable-programming-is-this-why-we-switched-to-pytorch/9589/5>

- The first neglected term is proportional to h .
- We therefore say that the one-sided approximation has **truncation error** $O(h)$.
- Making h smaller initially makes the estimate better.

Using both sides cancels an error term

The **symmetric difference quotient** samples on both sides of x :

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Expanding both terms around x gives

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(x) + O(h^4).$$

- The terms involving $f(x)$ and $f''(x)$ cancel.
- The truncation error is now $O(h^2)$ rather than $O(h)$.
- We spend one extra function evaluation to obtain a much better approximation for small h .

Why not make h as small as possible?

There are two competing sources of error:

Truncation error

The finite difference is only an approximation to the limit.

$$\begin{aligned} \text{one-sided: } & O(h) \\ \text{symmetric: } & O(h^2) \end{aligned}$$

It decreases as h gets smaller.

Roundoff error

The computer stores approximations to x , $x+h$ and the corresponding outputs.

$$\text{error} \sim O\left(\frac{\epsilon_{\text{mach}}}{h}\right)$$

It increases as h gets smaller.

There is a “sweet spot”: too large truncates; too small rounds away the signal.

Floating point numbers are not the reals

A floating point number has a finite number of bits for its sign, significand and exponent.

- Only a finite, non-uniformly spaced set of real numbers can be represented.
- The gap between adjacent representable values grows with the magnitude of the number.
- If h is too small, the rounded value of $x+h$ can be exactly x .
- Even before that happens, subtracting the nearby values $f(x+h)$ and $f(x)$ discards shared leading digits: **catastrophic cancellation**.

The consequence

The numerator becomes dominated by rounding noise just as we try to make the mathematical approximation more accurate.

Numerical differentiation needs double precision

32-bit floating point

- about 7 decimal digits
- $\epsilon_{\text{mach}} \approx 1.2 \times 10^{-7}$

64-bit floating point

- about 16 decimal digits
- $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$
- A rough choice for symmetric differences is $h \sim \sqrt[3]{\epsilon_{\text{mach}}}$, although the best value also depends on the scale and curvature of f .
- Single precision leaves very little room between truncation and roundoff error; half precision is worse still.
- For gradient checking, evaluate both the function and the gradients in **double precision**.

Finite differences do not scale to training

Let $\mathcal{L} : \mathbb{R}^p \rightarrow \mathbb{R}$ be a loss with p parameters.

- A one-sided numerical gradient needs $p + 1$ evaluations of $\mathcal{L}$.
- A symmetric numerical gradient needs $2p$ evaluations.
- A model with millions of parameters would therefore need millions of forward passes for *one* gradient step.
- The estimates also contain truncation and roundoff error.

So why learn this?

A slow, independent approximation is extremely useful for checking whether a fast derivative implementation is correct.

Finite differences make good gradient checks

For a scalar loss, compare an AD gradient g_{AD} with a symmetric finite-difference estimate g_{FD} :

$$\frac{\|g_{\text{AD}} - g_{\text{FD}}\|}{\max(1, \|g_{\text{AD}}\|, \|g_{\text{FD}}\|)}.$$

- Use double precision.
- Make the computation deterministic.
- Check a few coordinates or directions for very large models.

```
import torch.autograd as ag

x = x.double()
x.requires_grad_()
ag.gradcheck(f, (x,))
```

PyTorch numerical accuracy notes: https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html
<https://docs.pytorch.org/docs/stable/generated/torch.autograd.gradcheck.gradcheck.html>

How can you compute derivatives?

There are three ways to compute derivatives:

- Symbolically differentiate the function with respect to its parameters
 - by hand
 - using a CAS
- Make estimates using finite differences
- Use Automatic Differentiation

Problems

Numerical errors - could compound in deep nets; expensive

What is Automatic Differentiation (AD)?

Automatic Differentiation:

- applies the chain rule to the primitive operations executed by a program;
- computes derivatives alongside the program, accurate up to floating-point roundoff, rather than producing a closed-form expression;
- propagates tangents with values in forward mode, and adjoints back from outputs in reverse mode.

Lets think about differentiation and programming

Example 0.1 (Math).

$$\begin{aligned}x &=? \\ y &=? \\ a &= x \, y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example 0.2 (Code). `x = ?`

```
y = ?  
a = x * y  
b = sin(x)  
z = a + b
```

The Chain Rule of Differentiation

Recall the chain rule for a variable/function z that depends on y which depends on x :

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

In general, the chain rule can be expressed as:

$$\frac{\partial w}{\partial t} = \sum_i^N \frac{\partial w}{\partial u_i} \frac{\partial u_i}{\partial t} = \frac{\partial w}{\partial u_1} \frac{\partial u_1}{\partial t} + \frac{\partial w}{\partial u_2} \frac{\partial u_2}{\partial t} + \cdots + \frac{\partial w}{\partial u_N} \frac{\partial u_N}{\partial t}$$

where w is some output variable, and u_i denotes each input variable w depends on.

Applying the Chain Rule

Let's differentiate our previous expression with respect to some yet to be given variable t :

Expression

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

$$\begin{aligned}\frac{\partial x}{\partial t} &=? \\ \frac{\partial y}{\partial t} &=? \\ \frac{\partial a}{\partial t} &= x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t} \\ \frac{\partial b}{\partial t} &= \cos(x) \frac{\partial x}{\partial t} \\ \frac{\partial z}{\partial t} &= \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}\end{aligned}$$

If we substitute $t = x$ in the above we'll have an algorithm for computing $\partial z / \partial x$. To get $\partial z / \partial y$ we'd just substitute $t = y$.

Translating to code I

We could translate the previous expressions back into a program involving *differential variables* {**dx**, **dy**, ...} which represent $\partial x / \partial t, \partial y / \partial t, \dots$ respectively:

```
dx = ?
dy = ?
da = y * dx + x * dy
db = cos(x) * dx
dz = da + db
```

What happens to this program if we substitute $t = x$ into the math expression?

Translating to code II

```
dx = 1
dy = 0
da = y * dx + x * dy
db = cos(x) * dx
dz = da + db
```

The effect is remarkably simple: to compute $\partial z / \partial x$ we just seed the algorithm with **dx=1** and **dy=0**.

Translating to code III

```
dx = 0
dy = 1
da = y * dx + x * dy
db = cos(x) * dx
dz = da + db
```

To compute $\partial z / \partial y$ we just seed the algorithm with **dx=0** and **dy=1**.

Making Rules

- We've successfully computed the gradients for a specific function, but the process was far from automatic.
- We need to formalise a set of rules for translating a program that evaluates an expression into a program that evaluates its derivatives.
- We have actually already discovered 3 of these rules:

```
c = a + b    =>   dc = da + db
c = a * b    =>   dc = b * da + a * db
c = sin(a)   =>   dc = cos(a) * da
```

More rules

These initial rules:

```
c=a+b        =>   dc=da+db
c=a*b        =>   dc=b*da+a*db
c=sin(a)     =>   dc=cos(a)*da
```

can easily be extended further using multivariable calculus:

```
c=a-b        =>   dc=da-db
c=a/b        =>   dc=da/b-a*db/b**2
c=a**b       =>   dc=b*a**(b-1)*da+log(a)*a**b*db
c=cos(a)     =>   dc=-sin(a)*da
c=tan(a)     =>   dc=da/cos(a)**2
```

Forward Mode AD

- To translate using the rules we simply replace each primitive operation in the original program by its differential analogue.
- The order of computation remains unchanged: if a statement K is evaluated before another statement L , then the differential analogue of K is evaluated before the analogue statement of L .
- This is **Forward-mode Automatic Differentiation**.

Interleaving differential computation

A careful analysis of our original program and its differential analogue shows that its possible to interleave the differential calculations with the original ones:

```
x  = ?
dx = ?

y  = ?
dy = ?

a  = x * y
da = y * dx + x * dy

b  = sin(x)
db = cos(x) * dx

z  = a + b
dz = da + db
```

Dual Numbers

- This implies that we can keep track of the value and gradient at the same time.
- We can use a mathematical concept called a “Dual Number” to create a very simple direct implementation of AD.

Forward mode computes a JVP

For a function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, carry an input value and an input tangent together:

$$(\mathbf{x}, \dot{\mathbf{x}}) \mapsto (\mathbf{f}(\mathbf{x}), \dot{\mathbf{y}}), \quad \dot{\mathbf{y}} = \mathbf{J}_{\mathbf{f}}(\mathbf{x})\dot{\mathbf{x}}.$$

- The propagated tangent $\dot{\mathbf{y}}$ is a **Jacobian-vector product** (JVP).
- Choosing $\dot{\mathbf{x}} = \mathbf{e}_i$ gives the i th column of the Jacobian ($\mathbf{e}_i$ is the i -th standard basis vector).
- Choosing an arbitrary $\dot{\mathbf{x}} = \mathbf{v}$ gives the change in the output in direction $\mathbf{v}$, without constructing the Jacobian.

The JVP is a directional derivative

The JVP has an equivalent limit definition:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{v} = \left. \frac{d}{dt} \mathbf{f}(\mathbf{x} + t\mathbf{v}) \right|_{t=0}.$$

Which we could approximate numerically:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{v} \approx \frac{\mathbf{f}(\mathbf{x} + h\mathbf{v}) - \mathbf{f}(\mathbf{x} - h\mathbf{v})}{2h}.$$

Forward mode does something better

It propagates the same directional derivative through every primitive operation using the chain rule—accurate to floating point precision, with no choice of h .

Computing a JVP with PyTorch

```
import torch
from torch.func import jvp

def f(x):
    return torch.stack((x[0] * x[1], torch.sin(x[0])))

x = torch.tensor([2., 3.], dtype=torch.double)
v = torch.tensor([1., 0.], dtype=torch.double)

y, Jv = jvp(f, (x,), (v,))
```

- $\mathbf{y}$ is $\mathbf{f}(\mathbf{x})$.
- $\mathbf{Jv}$ is $\mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{v}$; here it is $[3, \cos(2)]^\top$.
- Each primal input has a tangent with the same shape.

PyTorch also exposes the dual tensors

The lower-level API makes the mechanism explicit:

```
from torch.autograd import forward_ad as fwAD

with fwAD.dual_level():
    dual_x = fwAD.make_dual(x, v)
    dual_y = f(dual_x)
    y, Jv = fwAD.unpack_dual(dual_y)
```

- `make_dual` pairs the primal $\mathbf{x}$ with the tangent $\mathbf{v}$.

<https://docs.pytorch.org/docs/stable/generated/torch.func.jvp.html>

- Ordinary PyTorch operations propagate both components.
- `unpack_dual` separates the output value from its JVP.

For routine JVPs, prefer the higher-level `torch.func.jvp`; the dual-tensor API is useful when you need finer control.